

Day 03: Mutable Sequences — Working with Lists

Welcome to Day 3! Today, we transition from immutable sequences (Strings and Tuples) to **Lists**, which are Python's primary **mutable** sequence type. Lists are incredibly versatile: they can grow or shrink dynamically, hold heterogeneous data types, and be modified directly in memory without creating new objects.

Section 1: List Fundamentals & Accessing Elements

1.1 What is a List?

A list is an ordered, indexed collection of items. In Python, lists are:

- **Mutable:** You can add, remove, or modify elements in-place.
- **Heterogeneous:** A single list can contain elements of different data types (e.g., integers, strings, other lists, booleans).
- **Dynamic:** Python handles resizing automatically.

[!NOTE] **Under the Hood (CPython Implementation):** In the standard CPython interpreter, lists are **not** implemented as linked lists. Instead, they are implemented as **variable-length dynamic arrays of object references (pointers)**.

- **Access Speed:** This contiguous array structure allows for very fast $O(1)$ constant-time lookup/modification of any element by index.
- **Memory Pre-allocation:** To avoid resizing the array on every `.append()`, Python overallocates capacity. As a result, appending elements has an **amortized** time complexity of $O(1)$.
- **Insertion/Deletion Costs:** Inserting or deleting elements from the beginning or middle of the list requires shifting all subsequent elements, yielding a time complexity of $O(n)$.

Syntax:

Lists are defined by enclosing comma-separated values inside square brackets `[...]`.

```
# An empty list
empty_list = []

# List of integers
numbers = [10, 20, 30, 40]

# List of mixed data types
mixed_list = ["Alice", 42, 3.14, True, None]

# Nested list (representing a 2D grid/matrix)
matrix = [
    [1, 2, 3],
```

```
[4, 5, 6]  
]
```

1.2 Accessing Elements (Indexing & Slicing)

Like Strings and Tuples, Lists are zero-indexed and support slicing.

1. Indexing

```
fruits = ["apple", "banana", "cherry", "date"]  
  
# Positive Indexing (Left-to-Right)  
print(fruits[0])    # Output: apple  
print(fruits[2])    # Output: cherry  
  
# Negative Indexing (Right-to-Left)  
print(fruits[-1])   # Output: date (Last element)  
print(fruits[-3])   # Output: banana
```

2. Slicing

Slicing extracts a sub-list using the syntax `list[start:stop:step]` (stop index is exclusive).

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]  
  
print(numbers[2:6])    # Output: [2, 3, 4, 5] (index 2 to 5)  
print(numbers[:4])     # Output: [0, 1, 2, 3] (start to index 3)  
print(numbers[5:])     # Output: [5, 6, 7, 8, 9] (index 5 to end)  
print(numbers[::2])    # Output: [0, 2, 4, 6, 8] (every second element)  
print(numbers[::-1])   # Output: [9, 8, 7, 6, 5, 4, 3, 2, 1, 0] (reverses  
list)
```

1.3 Memory Behavior: Aliasing vs. Copying (Crucial!)

Because lists are mutable, you must understand how Python manages variables pointing to them in memory.

1. Aliasing (Sharing References)

When you assign one list variable to another, Python does **not** create a copy of the list. Instead, both variables point to the **same object in memory**.

```
list1 = [1, 2, 3]  
list2 = list1  # list2 is now an alias for list1
```

```
list2.append(99)
print(list1) # Output: [1, 2, 3, 99] (list1 changed because list2 is list1!)
print(id(list1) == id(list2)) # Output: True
```

2. Shallow Copy (Outer-level Copy)

A **Shallow Copy** creates a new list container, but copies references to the items inside. If your list contains nested mutable objects (like nested lists), the copy and the original will still share the same nested sub-lists!

- **Methods:** Use `.copy()`, slice notation `[:]`, or the `list()` constructor.

```
# Shallow copy with simple values (works fine)
simple1 = [1, 2, 3]
simple2 = simple1.copy()
simple2.append(99)
print(simple1) # Output: [1, 2, 3] (Original is unaffected)

# Shallow copy with nested lists (shares reference to inner lists)
nested1 = [[1, 2], [3, 4]]
nested2 = nested1.copy()

# Modify the nested sub-list in the copy
nested2[0][0] = 99
print(nested1) # Output: [[99, 2], [3, 4]] (Original was modified!)
print(id(nested1[0]) == id(nested2[0])) # Output: True (Inner lists share the same reference)
```

3. Deep Copy (Recursive Copy)

A **Deep Copy** recursively copies all objects inside the list, creating entirely new, independent copies of all nested mutable elements.

- **Method:** Use Python's built-in `copy` module and call `copy.deepcopy()`.

```
import copy

nested1 = [[1, 2], [3, 4]]
nested2 = copy.deepcopy(nested1) # Recursively copies nested lists

# Modify the nested sub-list in the deep copy
nested2[0][0] = 99
print(nested1) # Output: [[1, 2], [3, 4]] (Original is completely safe and unaffected!)
```

```
print(nested2) # Output: [[99, 2], [3, 4]]
print(id(nested1[0]) == id(nested2[0])) # Output: False (Separate memory allocations)
```

Section 2: Modifying Lists & List Methods

Since lists are mutable, we can add, modify, or remove elements in-place.

2.1 Modifying Elements by Index

```
items = ["phone", "laptop", "tablet"]
items[1] = "desktop"
print(items) # Output: ['phone', 'desktop', 'tablet']
```

2.2 Adding Elements

- **.append(item)**: Adds an item to the end of the list.
- **.insert(index, item)**: Inserts an item at a specific index, shifting subsequent items to the right.
- **.extend(iterable)**: Appends all items of another iterable (like a list) to the end.

```
shopping = ["milk", "bread"]

# Append
shopping.append("eggs")
print(shopping) # Output: ['milk', 'bread', 'eggs']

# Insert
shopping.insert(1, "butter")
print(shopping) # Output: ['milk', 'butter', 'bread', 'eggs']

# Extend
snacks = ["chips", "cookies"]
shopping.extend(snacks)
print(shopping) # Output: ['milk', 'butter', 'bread', 'eggs', 'chips', 'cookies']
```

2.3 Removing Elements

- **.remove(item)**: Removes the first occurrence of **item** from the list. Raises a **ValueError** if the item is not found.
- **.pop(index)**: Removes and returns the item at **index**. If no index is provided, it removes and returns the **last** item.
- **del list[index]**: Deletes the element at the specified index or slice range.

- `.clear()`: Removes all elements, leaving the list empty.

```
tasks = ["code", "test", "deploy", "test"]

# Remove first occurrence
tasks.remove("test")
print(tasks) # Output: ['code', 'deploy', 'test']

# Pop last element
popped_item = tasks.pop()
print(f"Popped: {popped_item}") # Output: Popped: test
print(tasks) # Output: ['code', 'deploy']

# Pop by index
first_task = tasks.pop(0)
print(f"Popped index 0: {first_task}") # Output: Popped index 0: code
print(tasks) # Output: ['deploy']

# Del statement
numbers = [10, 20, 30, 40]
del numbers[1:3] # Deletes indices 1 and 2
print(numbers) # Output: [10, 40]
```

2.4 Searching and Sorting Operations

- `.index(item)`: Returns the index of the first occurrence of `item`. Raises `ValueError` if not present.
- `.count(item)`: Returns the number of times `item` appears in the list.
- `.sort()`: Sorts the list in-place (ascending order).
- `.reverse()`: Reverses the elements of the list in-place.

```
grades = [90, 75, 88, 75, 95]

print(grades.count(75)) # Output: 2
print(grades.index(88)) # Output: 2

# Sort in-place (ascending)
grades.sort()
print(grades) # Output: [75, 75, 88, 90, 95]

# Sort in-place (descending)
grades.sort(reverse=True)
print(grades) # Output: [95, 90, 88, 75, 75]

# Reverse in-place
grades.reverse()
print(grades) # Output: [75, 75, 88, 90, 95]
```

Section 3: List Operators & Helpers

3.1 Common List Operators

- **Concatenation (+)**: Joins two lists to form a **new** list.
- **Repetition (*)**: Repeats the list elements a specified number of times, returning a **new** list.
- **Membership (in / not in)**: Checks if an item exists inside a list, returning a Boolean.
- **In-Place Concatenation (+=)**: Appends the elements of another list to the existing list in-place (equivalent to `.extend()`).
- **In-Place Repetition (*=)**: Multiplies the elements of the list in-place.

Memory Comparison: Standard vs. In-Place Operators

Because lists are mutable, there is a major difference in memory handling between standard operators and their in-place shorthands:

```
# 1. Standard Concatenation vs. In-Place
lst = [1, 2]
print(id(lst))      # E.g., 4390192832

# Standard Concatenation (creates a NEW list object)
lst = lst + [3, 4]
print(id(lst))      # E.g., 4390195584 (Different ID – new list created!)

# In-Place Concatenation (modifies existing list in-place)
lst += [5, 6]
print(id(lst))      # E.g., 4390195584 (Same ID – modified in-place!)
```

```
# 2. Basic Operator Usage Examples
group1 = [1, 2]
group2 = [3, 4]

# Plus operator
combined = group1 + group2
print(combined)  # Output: [1, 2, 3, 4]

# Multiply operator
repeated = group1 * 3
print(repeated)  # Output: [1, 2, 1, 2, 1, 2]

# In-place repetition
group1 *= 2
print(group1)    # Output: [1, 2, 1, 2]

# Membership tests
colors = ["red", "green", "blue"]
```

```
print("red" in colors)      # Output: True
print("yellow" not in colors) # Output: True
```

Section 4: List Transformations & List Comprehensions

List comprehensions provide a clean, concise syntax for creating a new list by executing an operation on each element of an existing sequence.

4.1 Basic Syntax

The syntax for list comprehensions is written inside square brackets. Keywords are highlighted in **red**, and the optional filtering clause is enclosed in **[]**:

```
new_list = [expression for item in iterable [if condition]]
```

- **expression**: The output value or operation to perform on each item (e.g., `x ** 2`, `x.upper()`).
- **item**: The variable representing the current element from the iterable (e.g., `x`, `num`, `word`).
- **iterable**: The sequence or collection being looped over (e.g., `range()`, `list`, `string`).
- **[if condition]**: An **optional** filter. The item is only processed if this condition evaluates to `True`.

4.2 Comparison: Standard For Loop vs. List Comprehension

Let's create a list of squares of even numbers from 1 to 5.

Traditional Way:

```
squares = []
for x in range(1, 6):
    if x % 2 == 0:
        squares.append(x ** 2)
print(squares) # Output: [4, 16]
```

List Comprehension Way:

```
squares = [x ** 2 for x in range(1, 6) if x % 2 == 0]
print(squares) # Output: [4, 16]
```

4.3 Practical Use Cases of List Comprehensions

List comprehensions are not just syntactic sugar; they are widely used in Python for clean and efficient data processing. Here are the most common practical use cases:

1. Data Type Conversion (Type Casting)

Often, inputs read from a file or user terminal are received as strings. List comprehensions make it easy to parse them into numerical types.

```
string_numbers = ["10", "20", "30", "40"]
integers = [int(num) for num in string_numbers]
print(integers) # Output: [10, 20, 30, 40]
```

2. Text Cleaning & Normalization

You can clean lists of user strings (e.g., removing whitespace and converting to lowercase) in a single line.

```
raw_cities = [" Bangalore ", " MANGALORE", "chennai ", "Delhi"]
clean_cities = [city.strip().title() for city in raw_cities]
print(clean_cities) # Output: ['Bangalore', 'Mangalore', 'Chennai', 'Delhi']
```

3. Filtering Data

Extracting specific items from a list that match a logical condition.

```
emails = ["vinod@vinod.co", "kishori@acts.in", "student@gmail.com", "admin@vinod.co"]

# Keep only emails belonging to the 'vinod.co' domain
corporate_emails = [email for email in emails if email.endswith("@vinod.co")]
print(corporate_emails) # Output: ['vinod@vinod.co', 'admin@vinod.co']
```

4. Conditional Transformations (If-Else Expressions)

If you want to transform elements *and* include a fallback value when the condition is false, you can write the `if-else` statement **before** the `for` loop.

- **Syntax:** `[expr_if_true if condition else expr_if_false for item in iterable]`

```
scores = [45, 88, 30, 92, 50]
# Classify scores as "Pass" (>= 50) or "Fail" (< 50)
```



```
results = ["Pass" if score >= 50 else "Fail" for score in scores]
print(results) # Output: ['Fail', 'Pass', 'Fail', 'Pass', 'Pass']
```

5. Flattening a 2D List (Nested Loops)

You can flatten a multi-dimensional array (list of lists) into a flat 1D list using nested loop syntax inside the comprehension.

- **Syntax:** `[item for sublist in matrix for item in sublist]` (loops are written in the order they would be nested traditionally).

```
matrix = [[1, 2], [3, 4], [5, 6]]
flat_list = [num for row in matrix for num in row]
print(flat_list) # Output: [1, 2, 3, 4, 5, 6]
```

Section 5: Converting between Lists and Strings

Converting data between text strings and list collections is one of the most common scripting tasks.

5.1 Splitting Strings to Lists: `.split()`

The string method `.split(separator)` splits a single string into a list of strings based on the specified separator pattern. If no separator is provided, it splits by any whitespace.

```
csv_data = "apple,banana,cherry"
fruits_list = csv_data.split(",")
print(fruits_list) # Output: ['apple', 'banana', 'cherry']

sentence = "Python is awesome"
words = sentence.split() # Splits by spaces
print(words) # Output: ['Python', 'is', 'awesome']
```

5.2 Joining List items to Strings: `.join()`

The string method `separator.join(list)` joins a list of strings into a single string, inserting the separator string in between elements.

```
words = ["Python", "is", "awesome"]
sentence = " ".join(words)
print(sentence) # Output: Python is awesome

items = ["milk", "eggs", "bread"]
```

```
comma_separated = ", ".join(items)
print(comma_separated) # Output: milk, eggs, bread
```

Note: `.join()` only works if all elements inside the list are strings. If you have integers, cast them to strings first.

Section 6: Beginner Pitfalls

1. The `IndexError`

Trying to access or modify an index that does not exist in the list.

```
names = ["Alice", "Bob"]
# print(names[2]) # IndexError: list index out of range
```

Tip: Always use `len(list)` to verify boundaries.

2. Modifying a List while Iterating Over It

Modifying a list (adding or removing items) while looping over it using a `for` loop causes indices to shift, leading to skipped elements or logic errors.

```
# Dangerous Example (Avoid this):
nums = [1, 2, 3, 4]
for num in nums:
    if num % 2 == 0:
        nums.remove(num) # Modifying inside iteration!
```

Fix: Iterate over a copy of the list instead:

```
for num in nums.copy():
    if num % 2 == 0:
        nums.remove(num)
```